

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO



FACULTAD DE CIENCIAS DE LA COMPUTACIÓN Y DISEÑO DIGITAL CARRERA DE INGENIERÍA EN SOFTWARE

TEMA:

GA – EXPOSICIÓN SEGUNDO CORTE: HERRAMIENTAS CASE (UMBRELLO UML MODELLER)
Ciclo MDD: modelo UML, generación de código C#, verificación y roundtrip

DOCUMENTO INDIVIDUAL:

INVESTIGACIÓN INDIVIDUAL - ESTHER MUÑOZ

ASIGNATURA:

INGENIERÍA DE REQUISITOS (ISR-401)

DOCENTE:

ING. GLEISTON GUERRERO ULLOA, PhD.

ESTUDIANTE:

MUÑOZ QUIÑONEZ YERANICK ESTHER

RESPONSABILIDAD PRINCIPAL:

Fundamentación teórica, requisitos, comparación de herramientas, criterios éticos y fuentes científicas.

EQUIPO:

A

CURSO/PARALELO:

CUARTO NIVEL PARALELO “B”

SISTEMA PFC:

SISTEMA INTELIGENTE DE CONTROL Y SEGUIMIENTO DE TERAPIA FÍSICA

HERRAMIENTA MDD Y LENGUAJE OBJETIVO:

UMBRELLO UML MODELLER – C#

REPOSITORIO GITHUB:

<https://github.com/AngeloP2114/SISTEMA-DE-TERAPIA-EQUIPOA-UMBRELLO-UML/tree/main>

QUEVEDO – LOS RÍOS – ECUADOR

2026 – 2027 PPA

Índice

1. Resumen	1
2. Introducción	1
3. Model-Driven Engineering	1
3.1. Modelos como artefactos de primera clase.....	1
3.2. MDE, MDA y MDD	1
4. Niveles CIM, PIM y PSM	1
4.1. CIM: modelo independiente de la computación.....	2
4.2. PIM: modelo independiente de plataforma.....	2
4.3. PSM: modelo específico de plataforma	2
5. Transformaciones M2M y M2T	2
5.1. Transformaciones modelo-a-modelo.....	2
5.2. Transformaciones modelo-a-texto	2
6. Metamodelado, UML y perfiles	3
7. Trazabilidad desde requisitos	3
7.1. Requisitos funcionales relevantes.....	3
8. Descripción del subsistema	3
9. Estado del arte y desafíos de MDE	4
10. Justificación de Umbrello	4
10.1. Criterios de selección	4
10.2. Comparación con alternativas	5
11. Criterios éticos y de uso responsable	5
12. Relación entre la teoría y la demostración	5
13. Aporte personal de Esther	5
14. Conclusiones	6
Apéndice A: Matriz de fuentes científicas y normativas	6
Apéndice B: Glosario para la exposición	7
Apéndice C: Guion personal para Esther	7
Apéndice D: Preguntas que Esther debe poder responder	7

1. Resumen

Esta investigación desarrolla la fundamentación académica utilizada por el Equipo A para aplicar Ingeniería Dirigida por Modelos al Sistema Inteligente de Control y Seguimiento de Terapia Física. Se analizan los conceptos de MDE, MDA y MDD; los niveles CIM, PIM y PSM; las transformaciones modelo-a-modelo y modelo-a-texto; la función del metamodelado; la trazabilidad de requisitos y los desafíos de adopción industrial. Posteriormente, se relacionan dichos conceptos con el subsistema modelado y se justifica la selección de Umbrello UML Modeller frente a alternativas comerciales. La investigación también identifica los requisitos funcionales que sustentan las clases del diagrama, evalúa la utilidad de la herramienta para un proyecto académico y establece criterios éticos para comunicar con transparencia el alcance del código generado. La contribución de Esther proporciona el marco que permite interpretar la práctica no como un tutorial aislado, sino como una aplicación del desarrollo dirigido por modelos vinculada a los requisitos del PFC. Las fuentes utilizadas incluyen estándares OMG e ISO/IEC/IEEE y artículos científicos con DOI verificable, conforme a la rúbrica de evaluación.

2. Introducción

La complejidad de los sistemas de software ha impulsado estrategias que buscan elevar el nivel de abstracción y reducir la distancia entre los requisitos, el diseño y la implementación. La Ingeniería Dirigida por Modelos propone que los modelos sean artefactos centrales del proceso: expresan decisiones, pueden verificarse y sirven como entrada para transformaciones automatizadas. Esta visión contrasta con el uso meramente documental de UML, donde los diagramas se elaboran al inicio y se abandonan cuando comienza la programación [1, 2].

El PFC del Equipo A aborda el seguimiento de terapias físicas. El subsistema estudiado representa usuarios, pacientes, fisioterapeutas, planes, ejercicios, sesiones y evaluaciones. A través de Umbrello UML Modeller se pretende demostrar que la estructura definida en UML puede producir código fuente C#. Para interpretar correctamente esta demostración es necesario comprender qué automatiza MDD, qué información permanece fuera del generador y cómo se conserva la trazabilidad desde los requisitos.

La contribución de Esther se centra en la investigación conceptual, la identificación de requisitos, la justificación de la herramienta y la reflexión ética. Este trabajo da sustento a las decisiones técnicas tomadas por Frixon en el modelo y por Angelo en la generación.

3. Model-Driven Engineering

3.1. Modelos como artefactos de primera clase

Model-Driven Engineering (MDE) es un paradigma en el que los modelos son productos activos y procesables. No son únicamente imágenes; deben ajustarse a lenguajes y metamodelos que permitan analizarlos y transformarlos. Schmidt explica que MDE busca reducir la complejidad accidental mediante lenguajes de modelado y generadores adecuados al dominio [1].

Brambilla, Cabot y Wimmer destacan que el valor de MDE depende de la automatización y del papel de los modelos durante el ciclo de vida [3]. Un diagrama sin semántica procesable no basta. Por ello, la actividad exige un archivo nativo de Umbrello y no acepta herramientas de dibujo genérico. El modelo debe contener clases, tipos, visibilidad, operaciones y relaciones que el generador pueda interpretar.

3.2. MDE, MDA y MDD

Aunque los términos están relacionados, no son equivalentes:

- **MDE** es el paradigma amplio que coloca los modelos en el centro de la ingeniería.
- **MDA** es la arquitectura promovida por OMG para organizar modelos por niveles de independencia respecto de la plataforma.
- **MDD** es la aplicación práctica del paradigma al desarrollo, donde el modelo guía o produce parte de la implementación.

MDD no significa que todo el sistema se genere sin intervención. Selic advierte que su uso pragmático requiere seleccionar adecuadamente qué aspectos se modelan y qué decisiones permanecen en la programación [2]. En el proyecto de terapias físicas, MDD se aplica a la estructura orientada a objetos, pero no a la interfaz, la base de datos, la autenticación real ni las reglas clínicas completas.

4. Niveles CIM, PIM y PSM

La guía MDA organiza el desarrollo en niveles de abstracción [4].

4.1. CIM: modelo independiente de la computación

El CIM representa el problema, los actores y los objetivos sin detallar la solución tecnológica. Para el sistema de terapias físicas, el CIM incluye afirmaciones como:

- el fisioterapeuta diseña un plan para el paciente;
- el plan contiene ejercicios y sesiones;
- las sesiones generan información de progreso;
- el paciente debe poder consultar su evolución.

Estos enunciados expresan necesidades del dominio y pueden transformarse en requisitos funcionales.

4.2. PIM: modelo independiente de plataforma

El PIM representa la lógica y estructura del sistema sin ligarla completamente a C#, Java o una base de datos específica. El diagrama UML con las clases `Paciente`, `PlanTerapia` y `SesionTerapia` se aproxima a este nivel porque describe conceptos del dominio, atributos, operaciones y cardinalidades.

4.3. PSM: modelo específico de plataforma

El PSM incorpora decisiones de tecnología. Al seleccionar C# como lenguaje objetivo, los tipos y relaciones deben expresarse en construcciones compatibles con .NET. `DateTime`, `bool`, herencia mediante dos puntos y colecciones tipadas son decisiones asociadas al PSM. Umbrello produce código que acerca el PIM al PSM, aunque la transformación no cubre el proyecto completo.

Tabla 1: Aplicación de los niveles MDA al PFC

Nivel	Ejemplo en el proyecto	Evidencia
CIM	Necesidad de controlar tratamientos y progreso.	Actores y requisitos funcionales.
PIM	Clases, atributos, operaciones y relaciones UML.	Diagrama de clases de Umbrello.
PSM	Estructuras C# y tipos .NET.	Archivos <code>.cs</code> generados.

5. Transformaciones M2M y M2T

5.1. Transformaciones modelo-a-modelo

Una transformación M2M genera un modelo a partir de otro. QVT es el estándar de OMG asociado a este tipo de transformación [5]. Un ejemplo sería transformar un modelo conceptual independiente de plataforma en un modelo orientado a clases C# con estereotipos tecnológicos. El trabajo del Equipo A no implementa una transformación QVT explícita, pero utiliza su lógica conceptual al adaptar el modelo para un lenguaje objetivo.

5.2. Transformaciones modelo-a-texto

M2T genera texto a partir de un modelo. El código C# producido por Umbrello es un caso directo: una clase UML produce un archivo, un atributo produce un campo y una operación produce una firma. MOFM2T define principios para lenguajes de transformación de modelo a texto [6].

Tabla 2: Diferencia entre M2M y M2T

Tipo	Resultado	Ejemplo
M2M	Otro modelo conforme a un metamodelo.	PIM genérico transformado en PSM para .NET.
M2T	Texto ejecutable o documental.	Clases UML convertidas en archivos C#.

La distinción es importante porque evita llamar “MDD” a cualquier plantilla. La generación debe depender del contenido formal del modelo y producir una salida que conserve su semántica estructural.

6. Metamodelado, UML y perfiles

UML es un lenguaje estandarizado por OMG. Su especificación define elementos como clases, atributos, operaciones, asociaciones, generalización, agregación, composición y multiplicidad [7]. El metamodelo establece qué elementos existen y cómo pueden relacionarse. Umbrello no interpreta una caja dibujada como clase por apariencia; procesa un elemento de modelo creado como *Class*.

Los perfiles UML permiten extender el lenguaje con estereotipos y valores etiquetados. En plataformas empresariales pueden utilizarse estereotipos como *Entity*, *Service* o *Controller*. El subsistema del Equipo A no requirió perfiles especializados porque el objetivo era generar estructura de dominio. Esta decisión reduce la complejidad, pero limita el resultado: no se generan capas de persistencia, controladores web ni interfaz.

7. Trazabilidad desde requisitos

La trazabilidad permite identificar el origen y el impacto de los elementos del sistema. Gotel y Finkelstein describen la dificultad de mantener vínculos entre requisitos y artefactos posteriores cuando los procesos no registran relaciones explícitas [8]. En esta actividad, la cadena de trazabilidad se define como:

requisito → clase UML → archivo C# → miembro de código

7.1. Requisitos funcionales relevantes

Tabla 3: Requisitos del subsistema y su representación

ID	Requisito funcional	Clases relacionadas
RF-01	El sistema permitirá registrar y administrar pacientes.	Usuario, Paciente.
RF-02	El fisioterapeuta podrá evaluar a un paciente.	Fisioterapeuta, Paciente.
RF-03	El fisioterapeuta podrá crear un plan de terapia.	Fisioterapeuta, PlanTerapia.
RF-04	El plan incluirá uno o más ejercicios.	PlanTerapia, Ejercicio.
RF-05	El plan programará sesiones de terapia.	PlanTerapia, SesionTerapia.
RF-06	El sistema calculará el progreso de las sesiones.	SesionTerapia, Paciente.
RF-07	Se registrarán indicadores de progreso clínico.	EvaluacionProgreso.

ISO/IEC/IEEE 29148 establece que los requisitos deben ser claros, verificables y trazables [9]. La tabla anterior no sustituye una especificación completa, pero conecta el modelo con necesidades reconocibles del PFC.

8. Descripción del subsistema

El subsistema organiza el seguimiento básico de una terapia física. *Usuario* concentra información común de acceso; *Paciente* representa a la persona que recibe la intervención; *Fisioterapeuta* planifica y evalúa; *PlanTerapia* establece objetivos, fechas y estado; *Ejercicio* describe las actividades; *SesionTerapia* registra el desarrollo temporal; y *EvaluacionProgreso* almacena indicadores clínicos.

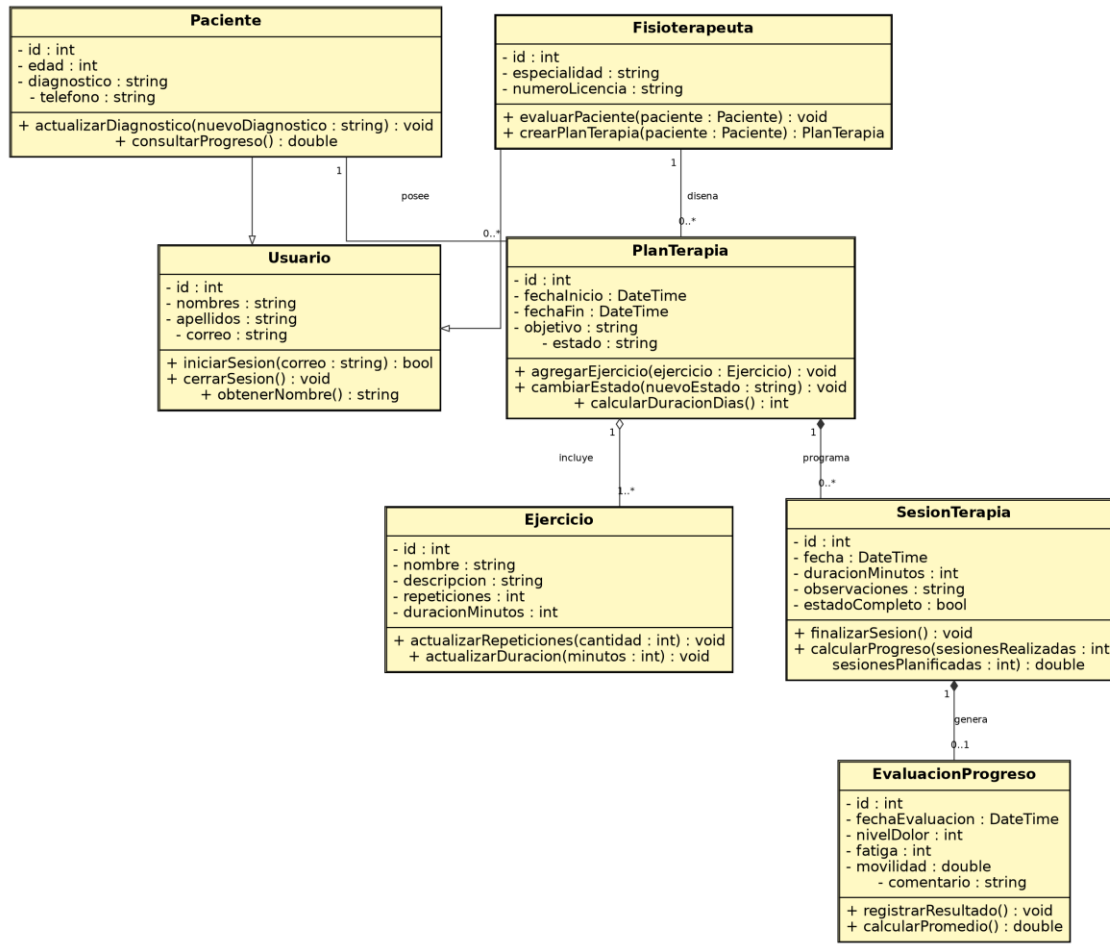


Figura 1: Modelo normalizado utilizado como base conceptual del subsistema.

El modelo supera el mínimo de seis clases establecido por la rúbrica y contiene varios tipos de relación. Su alcance se mantiene deliberadamente limitado: no intenta representar citas, facturación, inventario, notificaciones o expedientes completos. Esta delimitación permite que el equipo demuestre el ciclo MDD con un módulo comprensible.

9. Estado del arte y desafíos de MDE

Kahani et al. identifican una amplia diversidad de herramientas de transformación, con diferencias en lenguajes, metamodelos, automatización y extensibilidad [10]. Esta diversidad ofrece opciones, pero dificulta la interoperabilidad. Un modelo elaborado en una herramienta puede requerir ajustes para ser procesado por otra.

Bucchiarone et al. señalan desafíos como escalabilidad, evolución, experiencia de usuario, integración y adopción industrial [11]. Whittle et al. muestran que la práctica de MDE depende de factores organizacionales y de la capacidad de adaptar las técnicas a problemas concretos [12]. Estos resultados respaldan una conclusión importante: MDD aporta valor cuando el alcance de automatización es explícito y el equipo no espera una generación total del sistema.

Para el PFC, Umbrello se emplea en un escenario adecuado a sus capacidades: generar una estructura de clases pequeña y trazable. No se utiliza para producir una arquitectura empresarial completa ni para resolver automáticamente las reglas clínicas.

10. Justificación de Umbrello

10.1. Criterios de selección

La herramienta se evaluó según:

- capacidad de modelar clases y relaciones UML;
- generación real de C#;
- disponibilidad en Windows;

- licencia libre;
- archivo nativo conservable en GitHub;
- curva de aprendizaje compatible con el tiempo académico;
- posibilidad de demostrar el proceso en vivo.

KDE describe Umbrello como herramienta UML y generador de código, con soporte para varios lenguajes [13, 14]. Su licencia facilita el uso académico sin adquirir una suscripción. Además, el *Code Generation Wizard* hace visible la selección de clases y opciones, lo que fortalece la demostración.

10.2. Comparación con alternativas

Tabla 4: Comparación cualitativa de herramientas

Herramienta	Licencia	Generación	Observación
Umbrello	Libre	Estructural	Adecuada para aprendizaje y demostración reproducible.
Enterprise Architect	Comercial	Avanzada	Mayor personalización y roundtrip, pero más compleja y de pago [15].
Visual Paradigm	Comercial/freemium	Avanzada	Integración amplia; varias funciones dependen de la edición.
BOUML	Mixta según versión	Estructural	Ligera y rápida, con una interfaz menos familiar para principiantes.
Modelio	Libre/comercial	Modular	Amplía funciones mediante módulos y requiere configuración adicional.

Umbrello no fue seleccionada por ser la herramienta más potente, sino por el ajuste entre sus capacidades y los objetivos de la actividad. Esta justificación es más sólida que afirmar simplemente que “es gratis”.

11. Criterios éticos y de uso responsable

Aunque la herramienta no es un asistente de IA generativa, la actividad exige criterios éticos porque existe riesgo de confundir la autoría y el alcance del código. Los principios aplicados son:

1. **Transparencia:** indicar qué archivos y miembros provienen de Umbrello.
2. **No suplantación:** no presentar `Program.cs` ni lógica manual como salida automática.
3. **Revisión humana:** compilar, inspeccionar y probar antes de integrar el código.
4. **Responsabilidad:** los errores del generador o del modelo no se transfieren a la herramienta; el equipo debe corregirlos.
5. **Privacidad:** no utilizar datos clínicos reales en una demostración académica sin autorización.
6. **Trazabilidad:** conservar modelo, capturas, código e historial de commits.

En un sistema de salud, estas precauciones son especialmente relevantes. La estructura generada no debe interpretarse como validación clínica ni como garantía de seguridad.

12. Relación entre la teoría y la demostración

Tabla 5: Conceptos investigados y evidencia práctica

Concepto	Aplicación en el trabajo
MDE	El modelo se conserva como artefacto central del repositorio.
MDA	El UML representa un PIM y C# introduce decisiones de PSM.
MDD	Parte de la implementación se obtiene desde el modelo.
M2T	Las clases se convierten en archivos <code>.cs</code> .
Metamodelo UML	Las relaciones y elementos son objetos de modelo, no dibujos.
Trazabilidad	Requisitos, clases y archivos se relacionan mediante tablas.
Roundtrip controlado	Un cambio UML se comprueba después de regenerar.

La tabla muestra por qué la exposición no debe reducirse a los pasos del asistente. La práctica es la evidencia de conceptos previamente investigados.

13. Aporte personal de Esther

El aporte individual comprende:

1. investigación de MDE, MDA y MDD;

2. explicación de CIM, PIM y PSM;
3. diferenciación entre M2M y M2T;
4. revisión de UML, metamodelos y perfiles;
5. investigación del estado del arte y desafíos de adopción;
6. identificación de actores y requisitos funcionales;
7. justificación de Umbrello frente a alternativas;
8. formulación de criterios éticos y de transparencia;
9. apoyo a las referencias científicas en formato IEEE;
10. preparación del segmento teórico de la exposición.

Esta contribución proporciona la base académica que conecta la demostración técnica con la Ingeniería de Requisitos y los criterios de evaluación.

14. Conclusiones

MDD ofrece un mecanismo para mantener una relación más directa entre los requisitos, el modelo y la implementación, pero su éxito depende del alcance y la calidad de los artefactos. El proyecto de terapias físicas utiliza un modelo de dominio suficientemente representativo para generar una línea base en C#, sin afirmar que toda la aplicación fue automatizada.

La investigación confirma que Umbrello es una elección apropiada para una práctica académica por su licencia, accesibilidad, modelado UML y generación estructural. Su principal limitación es la falta de una sincronización avanzada y de lógica de negocio automática. Por ello, el equipo debe mantener control de versiones, separar el código manual y verificar cada salida.

El aporte teórico permite interpretar la práctica como una transformación M2T vinculada a un PIM y un PSM, con trazabilidad hacia requisitos funcionales. Esta comprensión es necesaria para responder preguntas del docente y demostrar que el equipo no se limitó a seguir un tutorial.

Apéndice A

Matriz de fuentes científicas y normativas

La siguiente matriz resume cómo se utilizó cada fuente. Esta evidencia facilita demostrar que la fundamentación no procede de blogs ni de definiciones aisladas.

Tabla 6: Fuentes utilizadas en la investigación de Esther

Fuente	Aporte conceptual	Aplicación en el documento
Schmidt (2006)	Definición de MDE y abstracción.	Explicación del papel activo de los modelos.
Selic (2003)	Aplicación pragmática de MDD.	Delimitación de lo que se genera y lo que permanece manual.
Brambilla et al. (2017)	Proceso práctico de ingeniería dirigida por modelos.	Relación entre modelos, transformaciones y código.
Whittle et al. (2014)	Estado de la práctica.	Análisis de factores organizacionales y adopción.
Bucchiarone et al. (2020)	Desafíos de investigación.	Limitaciones de interoperabilidad, evolución y escalabilidad.
Kahani et al. (2019)	Clasificación de herramientas.	Contexto del ecosistema de transformación.
Gotel y Finkelstein (1994)	Problema de trazabilidad.	Cadena requisito–modelo–código.
OMG MDA Guide	CIM, PIM y PSM.	Organización de niveles de abstracción.
OMG UML 2.5.1	Semántica de elementos UML.	Interpretación formal del diagrama de clases.
OMG MOFM2T	Transformaciones modelo-a-texto.	Clasificación de la generación C# de Umbrello.
OMG QVT	Transformaciones modelo-a-modelo.	Diferenciación frente a M2T.

Fuente	Aporte conceptual	Aplicación en el documento
ISO/IEC/IEEE 29148	Calidad y trazabilidad de requisitos.	Formulación de requisitos funcionales.

Apéndice B

Glosario para la exposición

MDE: paradigma general en el que los modelos son artefactos de primera clase.

MDA: propuesta de OMG que separa CIM, PIM y PSM.

MDD: aplicación de MDE para conducir el desarrollo y obtener implementaciones.

CIM: visión del dominio independiente de decisiones computacionales.

PIM: modelo de la lógica independiente de una plataforma concreta.

PSM: modelo adaptado a una plataforma, en este caso C#/.NET.

M2M: transformación cuyo resultado es otro modelo.

M2T: transformación cuyo resultado es texto, como código fuente.

Metamodelo: modelo que define la estructura válida de un lenguaje de modelado.

Trazabilidad: capacidad de seguir un requisito hasta diseño, código y pruebas.

Forward engineering: producción de código a partir del modelo.

Roundtrip controlado: cambio del modelo, regeneración y comprobación del cambio.

Apéndice C

Guion personal para Esther

La intervención puede durar entre tres y cuatro minutos y debe enlazar la teoría con el caso práctico:

1. explicar que MDE convierte al modelo en un artefacto procesable;
2. distinguir MDA y MDD sin presentarlos como sinónimos;
3. relacionar el diagrama con un PIM y el código C# con un PSM;
4. identificar la generación como transformación M2T;
5. justificar Umbrello por ajuste al PFC, licencia, Windows y C#;
6. mencionar una alternativa y explicar por qué no fue priorizada;
7. cerrar con criterios éticos: transparencia, revisión y separación del código manual.

Apéndice D

Preguntas que Esther debe poder responder

1. **¿MDE, MDA y MDD son iguales?** No. MDE es el paradigma, MDA es la arquitectura de OMG y MDD es la aplicación al desarrollo.
2. **¿La práctica contiene una transformación M2M?** No de forma explícita; la evidencia principal es M2T, aunque el paso PIM–PSM sirve como interpretación conceptual.
3. **¿Por qué Umbrello es admisible?** Porque genera código verificable desde elementos UML y conserva un modelo nativo.
4. **¿Por qué no basta con ser gratuito?** La selección debe responder al lenguaje, plataforma, licencia, curva de aprendizaje y necesidades del PFC.
5. **¿Qué riesgo ético existe?** Presentar código manual o incompleto como si fuera una aplicación generada y validada automáticamente.

Referencias

- [1] D. C. Schmidt, “Guest Editor’s Introduction: Model-Driven Engineering,” *Computer*, vol. 39, n.º 2, págs. 25-31, 2006. DOI: 10.1109/MC.2006.58.
- [2] B. Selic, “The Pragmatics of Model-Driven Development,” *IEEE Software*, vol. 20, n.º 5, págs. 19-25, 2003. DOI: 10.1109/MS.2003.1231146.
- [3] M. Brambilla, J. Cabot y M. Wimmer, *Model-Driven Software Engineering in Practice*, 2.ª ed. Morgan & Claypool Publishers, 2017. DOI: 10.2200/S00751ED2V01Y201701SWE001.
- [4] Object Management Group, “Model Driven Architecture (MDA) Guide, Revision 2.0,” Object Management Group, inf. téc. ormsc/14-06-01, 2014. dirección: <https://www.omg.org/cgi-bin/doc?ormsc/14-06-01>.

- [5] Object Management Group, “Meta Object Facility 2.0 Query/View/Transformation Specification, Version 1.3,” Object Management Group, inf. téc. formal/16-06-03, 2016. dirección: <https://www.omg.org/spec/QVT/1.3>.
- [6] Object Management Group, “MOF Model to Text Transformation Language, Version 1.0,” Object Management Group, inf. téc. formal/2008-01-16, 2008. dirección: <https://www.omg.org/spec/MOFM2T/1.0>.
- [7] Object Management Group, “OMG Unified Modeling Language (OMG UML), Version 2.5.1,” Object Management Group, inf. téc. formal/17-12-05, 2017. dirección: <https://www.omg.org/spec/UML/2.5.1>.
- [8] O. C. Z. Gotel y A. C. W. Finkelstein, “An Analysis of the Requirements Traceability Problem,” en *Proceedings of the First International Conference on Requirements Engineering*, 1994, págs. 94-101. DOI: 10.1109/ICRE.1994.292398.
- [9] ISO/IEC/IEEE, “ISO/IEC/IEEE 29148:2018: Systems and Software Engineering – Life Cycle Processes – Requirements Engineering,” inf. téc., 2018. dirección: <https://www.iso.org/standard/72089.html>.
- [10] N. Kahani, M. Bagherzadeh, J. R. Cordy, J. Dingel y D. Varro, “Survey and Classification of Model Transformation Tools,” *Software and Systems Modeling*, vol. 18, n.º 4, págs. 2361-2397, 2019. DOI: 10.1007/s10270-018-0665-6.
- [11] A. Bucchiarone, J. Cabot, R. F. Paige y A. Pierantonio, “Grand Challenges in Model-Driven Engineering: An Analysis of the State of the Research,” *Software and Systems Modeling*, vol. 19, n.º 1, págs. 5-13, 2020. DOI: 10.1007/s10270-019-00773-6.
- [12] J. Whittle, J. Hutchinson y M. Rouncefield, “The State of Practice in Model-Driven Engineering,” *IEEE Software*, vol. 31, n.º 3, págs. 79-85, 2014. DOI: 10.1109/MS.2013.65.
- [13] KDE Community. “Umbrello UML Modeller,” visitado 25 de jul. de 2026. dirección: <https://apps.kde.org/umbrello/>.
- [14] KDE Documentation Team. “Umbrello UML Modeller Handbook: Code Import and Code Generation,” visitado 25 de jul. de 2026. dirección: https://docs.kde.org/trunk_kf6/en/umbrello/umbrello/code-import-generation.html.
- [15] Sparx Systems. “Code Engineering with Enterprise Architect,” visitado 25 de jul. de 2026. dirección: <https://sparxsystems.com/enterprise-architect/code-engineering/code-engineering.html>.